

NLP Project : "RL-based Finetuning of Prompting LLM for Improved ICL Performance"

Nitin Kumar Rajesh (MT2024103)

Akash Chaudhari (MT2024012)

M B Ashish (MT2024085)

November 26, 2025

Abstract

In-context learning (ICL) is a powerful tool to the quality of an LLM's output. This is especially the case with large, publicly hosted LLMs and domain-specific LLMs. The main challenge with relying on ICL performance is that it relies solely on the user to enter a better prompt, which can be hard to quantify and involves trial and error.

Our project involves a prompting LLM to improve the user's prompt and feed it to a larger LLM. The prompting LLM has been fine-tuned through reinforcement learning with a task-specific dataset. The prompt is then fed to a larger LLM to generate a response.

The process we have developed starts from dataset curation, augmenting a typical instruction-tuning dataset with positive-negative response pairs, done entirely through automation

GitHub Repository: NLP-RL-Pipeline

1 Introduction

This project focuses on developing a small language model capable of generating optimized prompts for a larger, more capable LLM. The objective is to improve the quality, correctness, and reliability of downstream model outputs by learning how to craft effective prompts automatically. The workflow consists of three main stages:

1. Automated dataset creation
2. RL-finetuning of prompt model
3. Automated evaluation

2 Dataset creation

It was found that instruction tuning datasets are widely available, while RL-oriented positive/negative examples datasets were limited.

The first step involved constructing a training dataset that relates prompts to the quality of responses produced by a larger LLM. Medical-domain QA dataset and a generic instruction tuning dataset was used as the underlying task sources. For each question, multiple candidate prompts were generated, and the larger LLM produced responses to each prompt. These responses were then scored based on correctness or preference signals. The resulting dataset—containing question, prompt, model response, and reward—was structured to support both supervised initialization and RL fine-tuning of the prompt-generation model.

3 Reinforcement learning

A small LLM is trained using reinforcement learning to generate prompts that consistently lead to high-quality answers from the larger LLM.

3.1 Choice of prompting model

Qwen2.5-0.5B-Instruct was chosen as the prompting model. This model was chosen for the following reasons—

- Small size makes it suitable for usage in prompt generation tasks, alongside a larger model.
- Qwen 2.5 family claim to have improved ability to follow user prompts and generate relevant, structured outputs.
- Open source model. Transparent weights and architecture make it customizable.

3.2 Input processing steps

For reinforcement learning, each data entry follows the following format:

```
{
  "prompt": "<task specified>",
  "chosen": "<positive prompt>",
  "rejected": "<negative prompt>"
}
```

As mentioned in Section 1, positive and negative prompts are decided by an automated judge LLM during dataset creation.

Qwen2.5 models expect a chat template with roles specified. So for training, each data entry followed the following format:

```
[
  {"role": "system", "content": "<system_prompt>"},
  {"role": "user", "content": "<task specified>"},
  {"role": "assistant", "content": "<generated prompt>"}
]
```

- A common **"system"** prompt was given as follows:
"You are an expert LLM prompt engineer. Your goal is to generate clear, concise, and effective prompts for other large language models. Refine the 'Task Description' into a good prompt, adhering to best practices of prompt engineering."
- **"user"** prompt refers to the original task description.
- **"assistant"** prompt contains the chosen/rejected generated prompt examples. Note that "assistant" role entry is present for only "chosen" and "rejected" examples.

3.3 RL technique: Odds Ratio Preference Optimization

The Odds Ratio Preference Optimization (ORPO) framework is a reinforcement-learning-inspired method for preference-based fine-tuning LLMs. It is designed to directly optimize a model so that it prefers "chosen" responses over "rejected" ones, while retaining the stability and simplicity of supervised learning. Unlike traditional RLHF pipelines that require reward models and policy-gradient methods, ORPO merges preference optimization and supervised alignment into a single objective.

3.3.1 Theoretical Foundations

ORPO assumes access to a dataset of triplets:

$$(p, c, r)$$

where p is the prompt, c is the preferred (chosen) response, and r is the non-preferred (rejected) response.

The training objective encourages the model to assign higher likelihood to c than to r . This becomes a pairwise comparison problem framed through a ranking loss. ORPO integrates two components into one unified loss, allowing preference alignment during standard supervised fine-tuning:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \lambda \mathcal{L}_{\text{OR}}$$

1. Supervised Fine-Tuning Loss The supervised loss simply maximizes the likelihood of the chosen response:

$$\mathcal{L}_{\text{SFT}} = -\log \pi_{\theta}(c \mid p)$$

2. Odds Ratio Loss Instead of training a separate reward model, ORPO uses the log-odds ratio of the probabilities of the chosen and rejected responses. The odds for a token sequence x is defined as:

$$\text{Odds}(x) = \frac{\pi_{\theta}(x)}{1 - \pi_{\theta}(x)}$$

The odds ratio loss is then:

$$\mathcal{L}_{\text{OR}} = -\log \left(\frac{\text{Odds}(c)}{\text{Odds}(r)} \right) = -\log \left(\frac{\pi_{\theta}(c)(1 - \pi_{\theta}(r))}{(1 - \pi_{\theta}(c))\pi_{\theta}(r)} \right)$$

This objective pulls the model toward assigning relatively more probability mass to the preferred output.

3. Combined ORPO Objective The final loss is:

$$\mathcal{L}_{\text{ORPO}}(\theta) = -\log \pi_{\theta}(c \mid p) - \lambda \log \left(\frac{\pi_{\theta}(c \mid p) [1 - \pi_{\theta}(r \mid p)]}{[1 - \pi_{\theta}(c \mid p)] \pi_{\theta}(r \mid p)} \right)$$

Here, λ controls the contribution of preference alignment relative to supervised fitting.

3.3.2 Advantages of ORPO

ORPO is increasingly adopted because it simplifies and stabilizes preference-based fine-tuning:

- **No Reward Model Needed:** ORPO directly optimizes preferences without building a separate reward model, unlike classical RLHF.
- **No PPO / RL Instabilities:** It eliminates policy gradient steps, advantage estimation, KL penalties, and unstable reward scaling.
- **Single-Stage Training:** Preference optimization is embedded directly into supervised training, reducing complexity.
- **Lower Compute Requirements:** Training resembles standard supervised fine-tuning, making it accessible to smaller labs.
- **Monotonic Preference Improvement:** The odds-ratio objective encourages consistent ranking improvement of preferred samples.
- **Better Sample Efficiency:** ORPO requires fewer preference pairs for meaningful alignment progress.

3.3.3 Reasons for Choosing ORPO for This Project

The goal of this project is to train a small model to generate improved prompt suggestions for a larger LLM. ORPO is an ideal fit for several reasons:

- **Direct Preference Optimization for Prompts:** Since the task involves ranking “better” vs. “worse” prompts, ORPO maps naturally onto the dataset format.
- **High Stability on Small Datasets:** Prompt-improvement datasets are often small; ORPO handles this far better than PPO-based RLHF.
- **Efficient Experimentation:** ORPO training runs are fast, enabling multiple experiments without heavy compute requirements.
- **Encourages Model to Learn Prompt-Engineering Patterns:** The odds-ratio mechanism magnifies the difference between good and bad prompt formulations.
- **Better Alignment Without Overfitting:** The λ parameter allows controlling how strongly the model prioritizes chosen over rejected responses.

Overall, ORPO works as a simple, computationally efficient, and relatively stable method for aligning a small model toward generating strong improvement prompts.

3.4 Dataset usage details

The biology-oriented generated prompts dataset consisted of 4954 positive-negative pairs for training. It was split into 4211 training examples.

The remaining 743 original tasks were used for inference generation (positive-negative pairs were ignored for this section).

Dataset Split	Samples	Percentage	Notes
Train	4211	85%	Used for model training
Test	743	15%	Used for evaluation
Total	4954	100%	—

Table 1: Dataset distribution across train and test splits.

4 Inference generation

4.1 Prompt inference generation

Using the Qwen2.5-0.5B model, inferences were generated. This process was done *before and after* ORPO fine-tuning.

In this workflow, inferencing involves preparing prompts using the model’s designated chat template, tokenizing them efficiently, and generating outputs under controlled decoding parameters such as `max_new_tokens = 108`, `temperature = 0.3`, and `top_p = 0.9`. These parameters allow fine-grained control over creativity, determinism, and output length. By batching prompts and leveraging GPU acceleration, inference throughput can be improved significantly, enabling large-scale evaluation or automated generation tasks.

4.1.1 Sample output

A sample output is shown below the following input prompt:

”Develop a creative narrative about neural prosthetics”

The output for the **untrained Qwen model** is as follows: from Qwen2.5-0.5B was generated for the following prompt:

”Certainly! Here’s a creative narrative for a neural prosthetic story:” — Once upon a time, in a world where humans had lost their ability to move due to injury or disease, there was a group of scientists who were determined to find a way to restore movement to those who could no longer walk. They began by developing a new type of artificial nerve called ”neuroprostheses,” which would mimic the signals sent by damaged nerves. The story begins with a young woman named Ava, who has been paralyzed from the waist down”

The output for the **ORPO-finetuned Qwen** model for the same prompt shown below:

”Discuss the potential benefits and challenges of developing neural prosthetics in human physiology. Provide a detailed analysis using real-world examples and scientific evidence. How might neural prosthetics aid in treating neurological disorders such as stroke or spinal cord injury? Discuss the ethical considerations surrounding the use of neural prosthetics and their impact on society. Use empirical data from clinical trials and case studies to support your arguments”

It is evident from the above example that ORPO-finetuning has helped the model produce more grounded responses, relevant for the domain. This can be seen as an outlier case, where the word 'creative' threw the base Qwen model off track. The fine-tuned Qwen model was able to appropriately provide a relevant biology-oriented prompt.